STAT 417 Final Code Work

Group 8

2026-04-27

Initial data pull/clean

```
#read the data from the fed website
library(readr)
url <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATENSA"
unratensa_data <- read_csv(url)

#convert to time series
unratensa_ts <- ts(unratensa_data$UNRATENSA, start = c(1948, 1), frequency = 12)
#interpolate the NA values
library(forecast)
unratensa_ts_interp <- na.interp(unratensa_ts)

#summary stats
summary(unratensa_ts_interp)
```

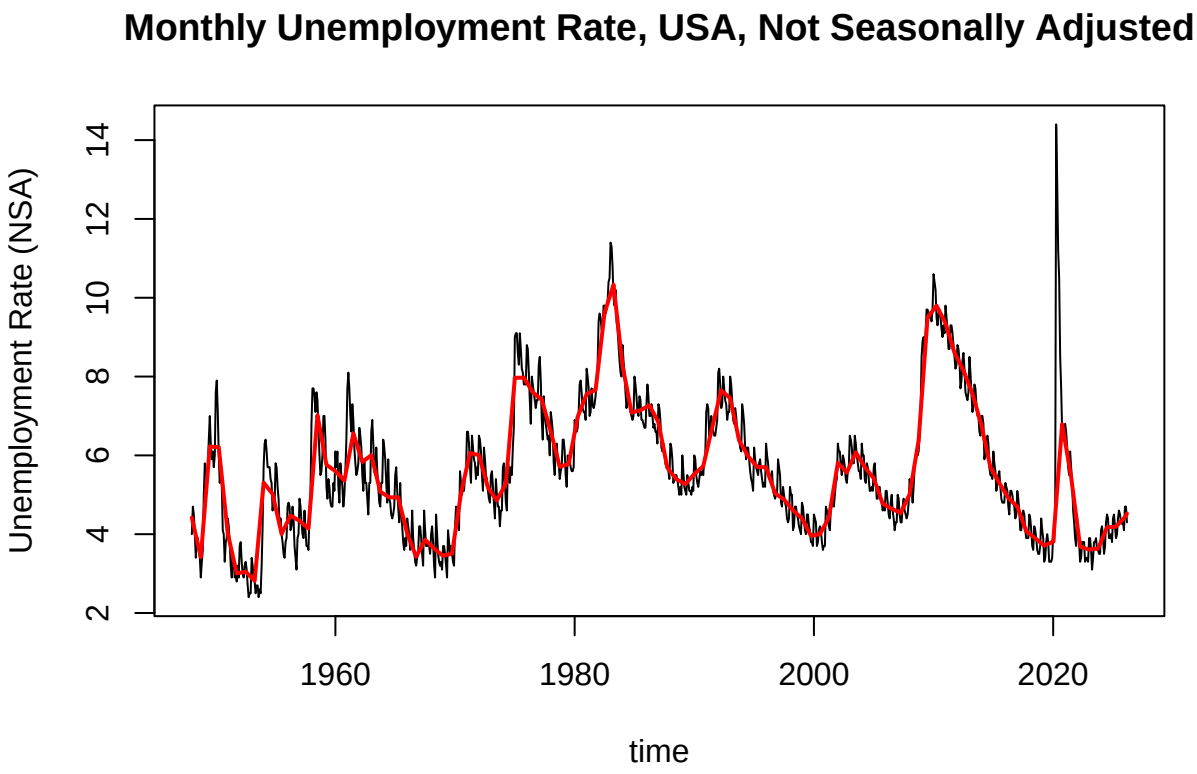
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.400   4.400   5.400   5.662   6.700  14.400
```

```
sd(unratensa_ts_interp)
```

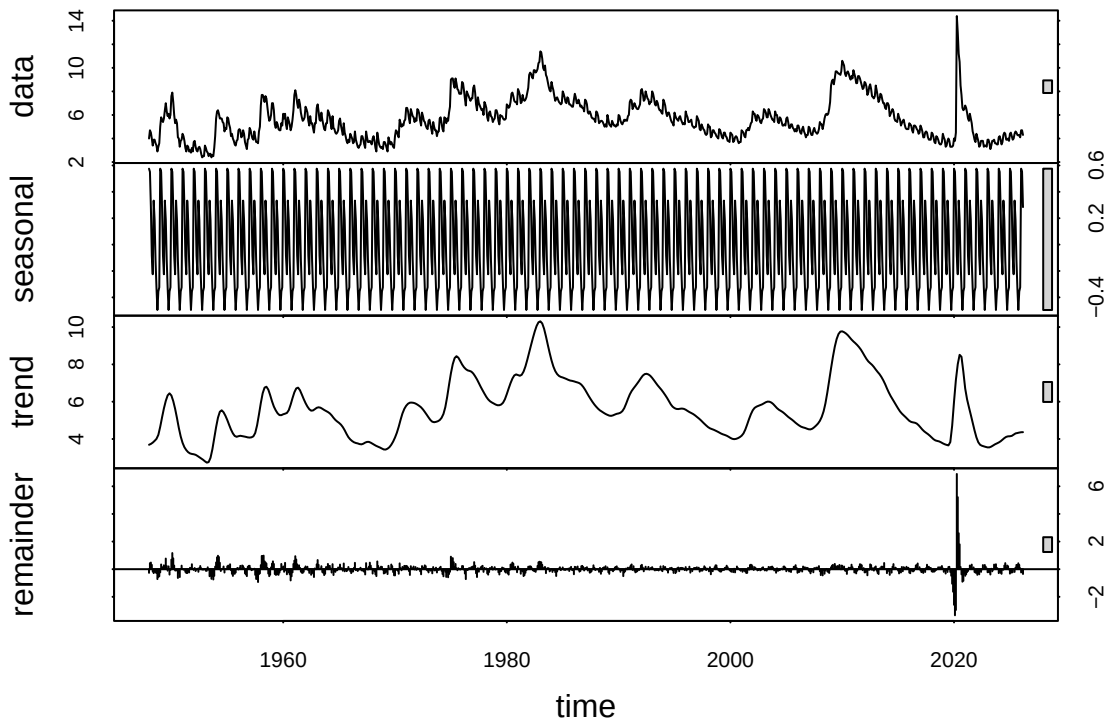
```
## [1] 1.744923
```

smoothing and stl

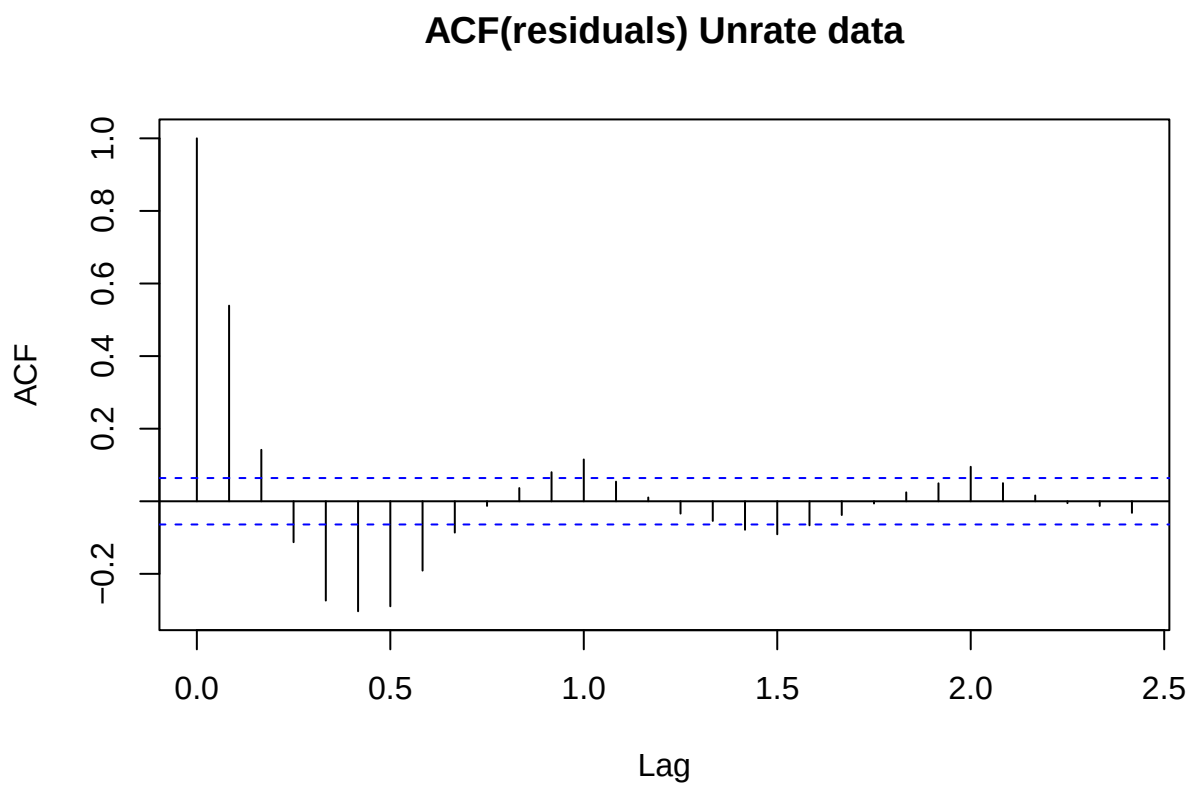
```
#full data set and smoothing
plot(unratensa_ts_interp, ylab="Unemployment Rate (NSA)", xlab = "time", col = "black", main = "Monthly
lines(lowess(unratensa_ts_interp, f=0.01), lwd=2, col="red")
```




```r
#stl analysis
unratensa_stl <- stl(unratensa_ts_interp, s.window = "periodic")
plot(unratensa_stl)
```




```r
season <- unratensa_stl$time.series[,1]
trend <- unratensa_stl$time.series[,2]
res <- unratensa_stl$time.series[,3]

acf(res, main = "ACF(residuals) Unrate data")
```



Looking at the entire data set, we can see seasonality in the tiny spikes.

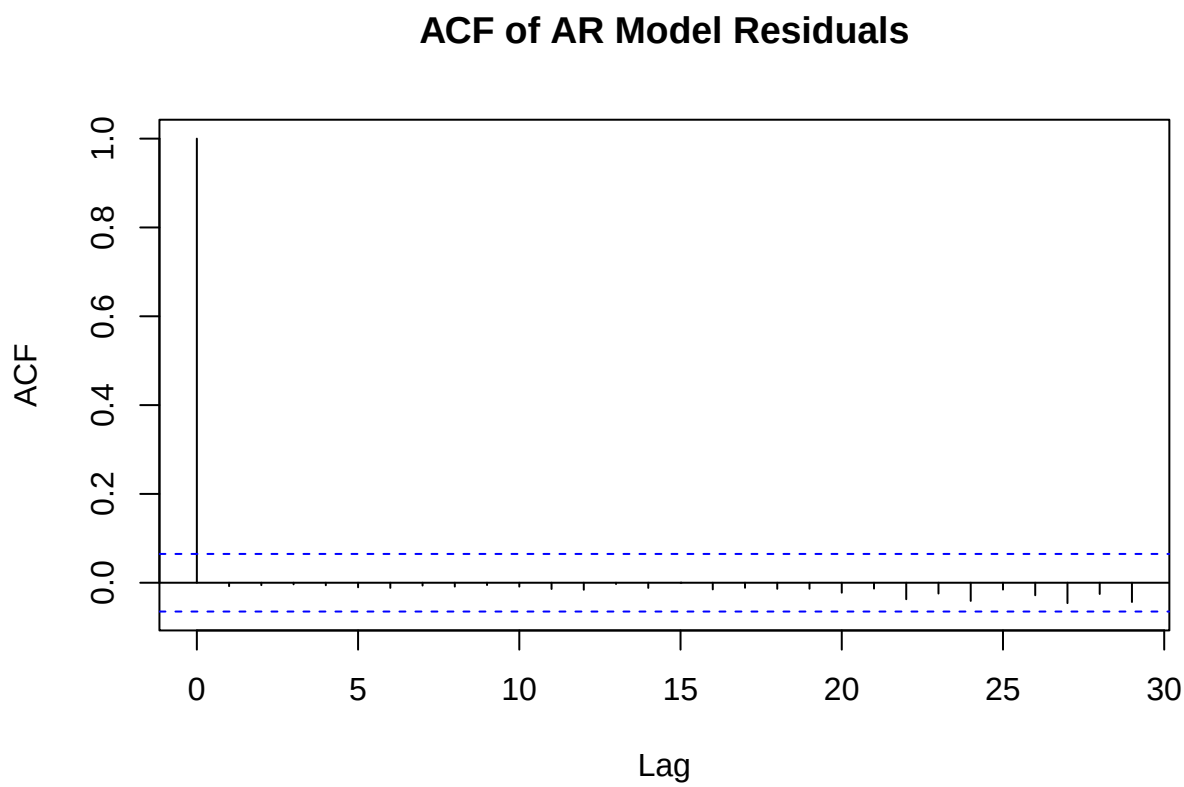
ACF reveals time dependence of the residuals, so we need to model the residuals.

Residual AR fit model

```
#get the order of the AR fit
fit_ar <- ar.yw(res, order.max = NULL)
fit_ar
```

```
##
## Call:
## ar.yw.default(x = res, order.max = NULL)
##
## Coefficients:
##       1        2        3        4        5        6        7        8
##  0.5390  -0.1878  -0.0777  -0.1762  -0.0508  -0.1600  -0.0535  -0.0702
##       9       10       11       12       13       14       15       16
## -0.0747  -0.0875  -0.0626   0.0171  -0.1280  -0.0235  -0.0851  -0.0188
##      17       18       19       20       21       22       23       24
## -0.0776  -0.0515  -0.0488  -0.0375  -0.0470  -0.0208  -0.0613   0.0722
##      25
## -0.0894
##
## Order selected 25  sigma^2 estimated as  0.1151
```

```r
# AR residuals
ar_resid <- ts(fit_ar$resid, start=start(unratensa_ts_interp), frequency=12)

# AR fitted values for the residual component
ar_fit <- res - ar_resid

# remove first p values when checking ACF because AR residuals begin with NA values
p <- fit_ar$order

acf(ar_resid[-c(1:p)],
    main = "ACF of AR Model Residuals")
```



Yule-Walker method selected an AR(25) model, suggesting that the random shocks have a long-lasting effect on the data.

Trend model

```r
time <- 1:length(trend)
dat.trend <- data.frame(trend = trend, time = time)

# trig fit
fit_trig <- lm(trend ~
                 cos(2*pi*time/12) + sin(2*pi*time/12),
               data = dat.trend)
summary(fit_trig)
```

```
##
## Call:
## lm(formula = trend ~ cos(2 * pi * time/12) + sin(2 * pi * time/12),
##     data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.9183 -1.2972 -0.1800  0.9667  4.6580
##
## Coefficients:
##                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)            5.660154   0.052206 108.419   <2e-16 ***
## cos(2 * pi * time/12) -0.004646   0.073870  -0.063    0.950
## sin(2 * pi * time/12) -0.009344   0.073792  -0.127    0.899
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.6 on 936 degrees of freedom
## Multiple R-squared:  2.139e-05,  Adjusted R-squared:  -0.002115
## F-statistic: 0.01001 on 2 and 936 DF,  p-value: 0.99
```

```r
# polynomial fits
fit_poly3 <- lm(trend ~ time + I(time^2) + I(time^3), data = dat.trend)

fit_poly5 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5), data = dat.trend)

fit_poly6 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6), data = dat.trend)

fit_poly7 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6) + I(time^7), data = dat.trend)

fit_poly8 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6) +
                  I(time^7) + I(time^8), data = dat.trend)

# summaries
summary(fit_poly3)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.3700 -1.0547 -0.2808  0.8407  3.9877
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.971e+00  1.878e-01  21.149  < 2e-16 ***
## time         7.943e-03  1.729e-03   4.594 4.94e-06 ***
## I(time^2)   -3.613e-06  4.273e-06  -0.846    0.398
## I(time^3)   -4.726e-09  2.988e-09  -1.581    0.114
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.433 on 935 degrees of freedom
## Multiple R-squared:  0.1989, Adjusted R-squared:  0.1963
## F-statistic: 77.37 on 3 and 935 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly5)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.4769 -1.0720 -0.1943  0.8512  3.7451
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.292e+00  2.727e-01  19.408  < 2e-16 ***
## time        -3.385e-02  5.851e-03  -5.786 9.85e-09 ***
## I(time^2)    3.066e-04  3.852e-05   7.960 4.99e-15 ***
## I(time^3)   -8.833e-07  1.038e-07  -8.507  < 2e-16 ***
## I(time^4)    1.051e-09  1.218e-10   8.629  < 2e-16 ***
## I(time^5)   -4.468e-13  5.155e-14  -8.667  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.38 on 933 degrees of freedom
## Multiple R-squared:  0.2586, Adjusted R-squared:  0.2546
## F-statistic: 65.08 on 5 and 933 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly6)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.2637 -1.1037 -0.2130  0.7824  3.8588
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.465e+00  3.151e-01  14.168  < 2e-16 ***
## time         2.738e-03  9.265e-03   0.296 0.767639
## I(time^2)   -8.141e-05  8.572e-05  -0.950 0.342497
## I(time^3)    7.656e-07  3.422e-07   2.237 0.025497 *
## I(time^4)   -2.237e-09  6.619e-10  -3.379 0.000757 ***
## I(time^5)    2.630e-12  6.113e-13   4.302 1.87e-05 ***
## I(time^6)   -1.091e-15  2.160e-16  -5.051 5.30e-07 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.362 on 932 degrees of freedom
## Multiple R-squared:  0.2783, Adjusted R-squared:  0.2737
## F-statistic: 59.91 on 6 and 932 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly7)
```

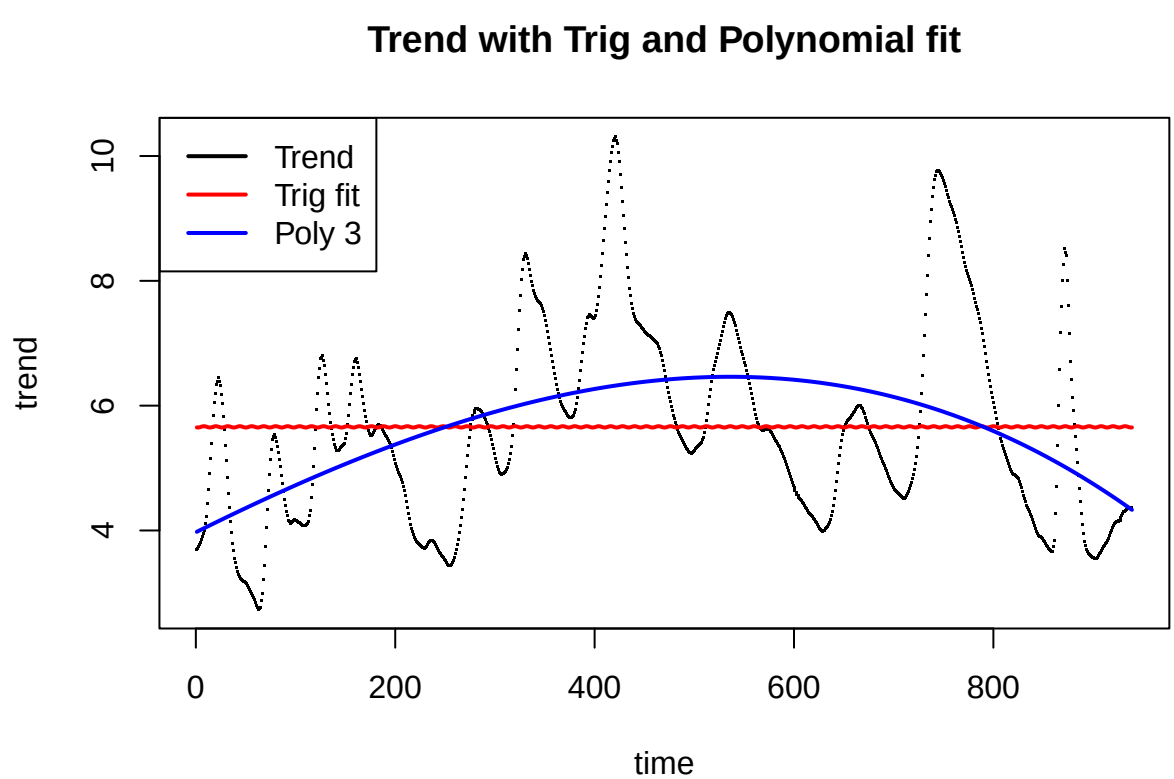
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.4090 -1.0560 -0.1070  0.8746  3.4647
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.787e+00  3.440e-01   8.102 1.69e-15 ***
## time         1.013e-01  1.326e-02   7.637 5.53e-14 ***
## I(time^2)   -1.490e-03  1.635e-04  -9.117  < 2e-16 ***
## I(time^3)    9.075e-06  8.968e-07  10.120  < 2e-16 ***
## I(time^4)   -2.652e-08  2.522e-09 -10.516  < 2e-16 ***
## I(time^5)    3.982e-11  3.785e-12  10.521  < 2e-16 ***
## I(time^6)   -2.966e-14  2.880e-15 -10.298  < 2e-16 ***
## I(time^7)    8.683e-18  8.732e-19   9.944  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.296 on 931 degrees of freedom
## Multiple R-squared:  0.3476, Adjusted R-squared:  0.3427
## F-statistic: 70.87 on 7 and 931 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly8)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7865 -0.9008 -0.0317  0.6954  4.1089
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.097e+00  3.534e-01  14.425  < 2e-16 ***
## time        -7.246e-02  1.728e-02  -4.194 3.01e-05 ***
## I(time^2)    1.725e-03  2.731e-04   6.317 4.12e-10 ***
## I(time^3)   -1.594e-05  1.960e-06  -8.132 1.34e-15 ***
## I(time^4)    7.314e-08  7.463e-09   9.800  < 2e-16 ***
```

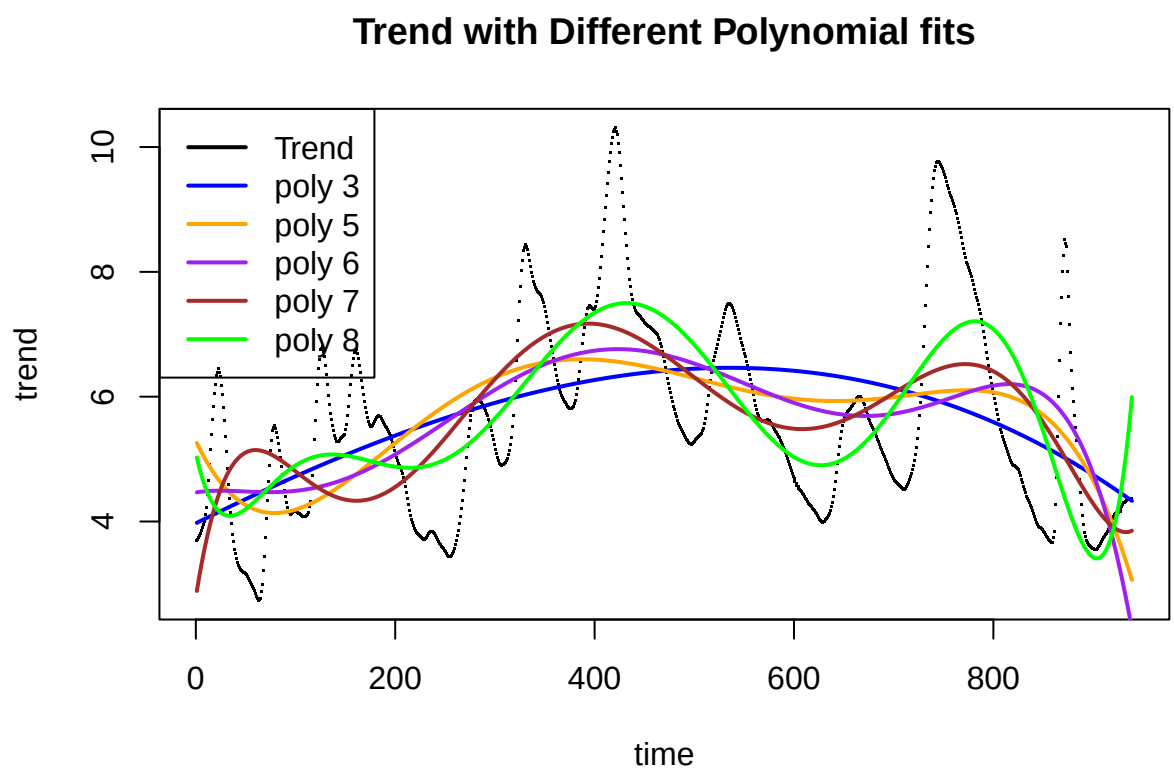
```
## I(time^5)   -1.805e-10  1.608e-11 -11.231  < 2e-16 ***
## I(time^6)    2.437e-13  1.966e-14  12.399  < 2e-16 ***
## I(time^7)   -1.694e-16  1.271e-17 -13.323  < 2e-16 ***
## I(time^8)    4.735e-20  3.374e-21  14.033  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.178 on 930 degrees of freedom
## Multiple R-squared:  0.4616, Adjusted R-squared:  0.457
## F-statistic: 99.68 on 8 and 930 DF,  p-value: < 2.2e-16
```

```r
# compare trig vs polynomial
plot(time, trend, pch=".", main="Trend with Trig and Polynomial fit")
lines(time, fitted(fit_trig), col="red", lwd=2)
lines(time, fitted(fit_poly3), col="blue", lwd=2)
legend("topleft",
       legend=c("Trend", "Trig fit","Poly 3"),
       col=c("black", "red", "blue"),
       lwd=2)
```




```r
# compare polynomial degrees
plot(time, trend, pch=".", main="Trend with Different Polynomial fits")

lines(time, fitted(fit_poly3), col="blue", lwd=2)
lines(time, fitted(fit_poly5), col="orange", lwd=2)
lines(time, fitted(fit_poly6), col="purple", lwd=2)
```

```r
lines(time, fitted(fit_poly7), col="brown", lwd=2)
lines(time, fitted(fit_poly8), col="green", lwd=2)

legend("topleft",
       legend=c("Trend", "poly 3","poly 5","poly 6","poly 7","poly 8"),
       col=c("black", "blue", "orange","purple","brown","green"),
       lwd=2)
```




```r
# AIC/BIC selection criteria
AIC(fit_poly3, fit_poly5, fit_poly6, fit_poly7, fit_poly8)
```

```
##           df      AIC
## fit_poly3  5 3345.936
## fit_poly5  7 3277.219
## fit_poly6  8 3253.863
## fit_poly7  9 3161.072
## fit_poly8 10 2982.724
```

```r
BIC(fit_poly3, fit_poly5, fit_poly6, fit_poly7, fit_poly8)
```

```
##           df      BIC
## fit_poly3  5 3370.161
## fit_poly5  7 3311.132
## fit_poly6  8 3292.622
```

```
## fit_poly7  9 3204.675
## fit_poly8 10 3031.172
```

The trig model for trend is not significant, with R^2 almost zero, which is expected.

AIC/BIC decrease as the polynomial degree increases, but the polynomial fit 8 seems to favor short term fluctuations rather than the overall structure. It is very unstable at the edge as well.

Seasonal model

```r
time <- 1:length(season)

# month index 1-12 repeating
month <- cycle(unratensa_ts_interp)

dat.season <- data.frame(
  season = as.numeric(season),
  time = time,
  month = as.numeric(month)
)

# trig fit, 1 harmonic
fit_season1 <- lm(season ~
                    cos(2*pi*month/12) + sin(2*pi*month/12),
                  data = dat.season)

# trig fit, 3 harmonics
fit_season2 <- lm(season ~
                    cos(2*pi*month/12) + sin(2*pi*month/12) +
                    cos(4*pi*month/12) + sin(4*pi*month/12) +
                    cos(6*pi*month/12) + sin(6*pi*month/12),
                  data = dat.season)

# check model summaries
summary(fit_season1)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * month/12) + sin(2 * pi * month/12),
##     data = dat.season)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.41997 -0.20964 -0.01918  0.29451  0.46633
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             0.0008124  0.0092394   0.088 0.929949
## cos(2 * pi * month/12) -0.0495083  0.0130733  -3.787 0.000162 ***
## sin(2 * pi * month/12)  0.3026840  0.0130595  23.177  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.2831 on 936 degrees of freedom
## Multiple R-squared:  0.3706, Adjusted R-squared:  0.3693
## F-statistic: 275.6 on 2 and 936 DF,  p-value: < 2.2e-16
```

```r
summary(fit_season2)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * month/12) + sin(2 * pi * month/12) +
##     cos(4 * pi * month/12) + sin(4 * pi * month/12) + cos(6 *
##     pi * month/12) + sin(6 * pi * month/12), data = dat.season)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.157399 -0.081057  0.004208  0.094241  0.170761
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             6.604e-05  3.211e-03   0.021 0.983597
## cos(2 * pi * month/12) -5.051e-02  4.544e-03 -11.117  < 2e-16 ***
## sin(2 * pi * month/12)  3.016e-01  4.539e-03  66.453  < 2e-16 ***
## cos(4 * pi * month/12)  3.671e-02  4.541e-03   8.084 1.94e-15 ***
## sin(4 * pi * month/12)  3.400e-01  4.541e-03  74.875  < 2e-16 ***
## cos(6 * pi * month/12) -1.525e-01  4.544e-03 -33.559  < 2e-16 ***
## sin(6 * pi * month/12) -1.513e-02  4.539e-03  -3.333 0.000894 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0984 on 932 degrees of freedom
## Multiple R-squared:  0.9243, Adjusted R-squared:  0.9238
## F-statistic:  1897 on 6 and 932 DF,  p-value: < 2.2e-16
```

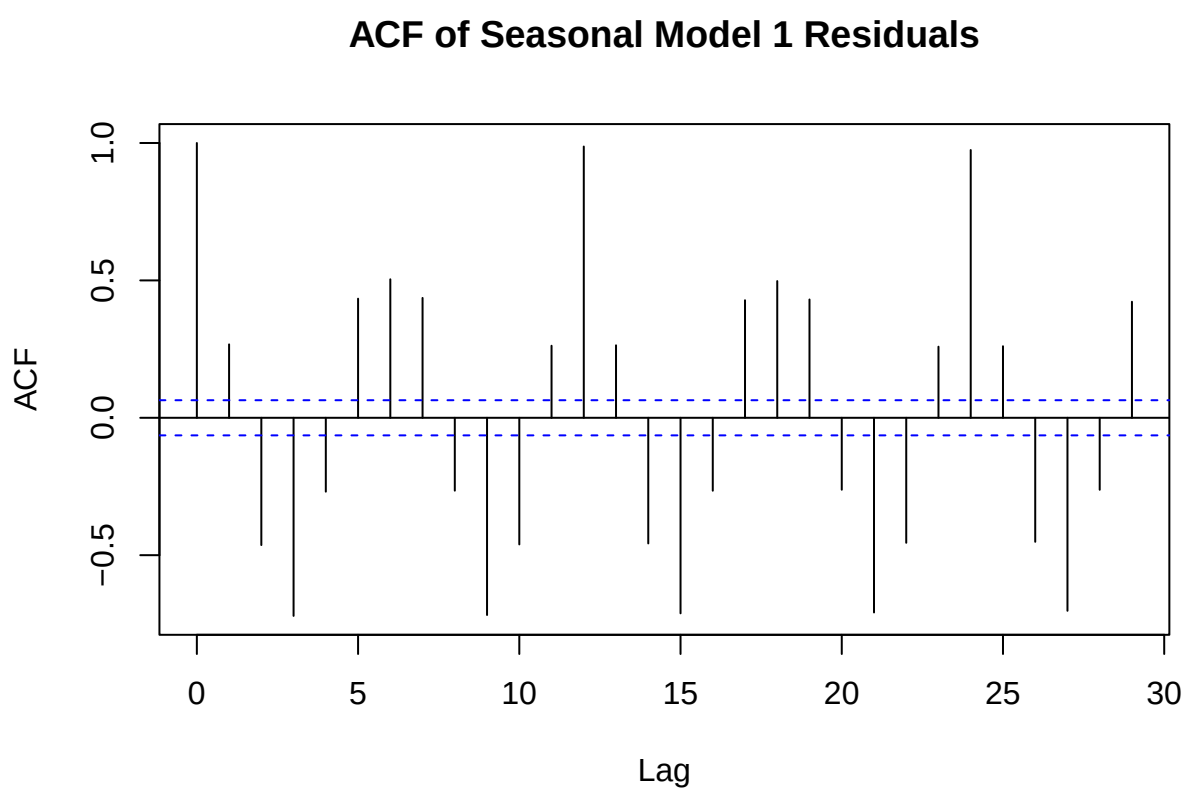
```r
# fitted seasonal values
season_fit1 <- fitted(fit_season1)
season_fit2 <- fitted(fit_season2)

# residuals from seasonal fits
season_resid1 <- residuals(fit_season1)
season_resid2 <- residuals(fit_season2)

# AIC/BIC comparison
AIC(fit_season1, fit_season2)
```

```
##             df        AIC
## fit_season1  4   299.9439
## fit_season2  8 -1680.7950
```

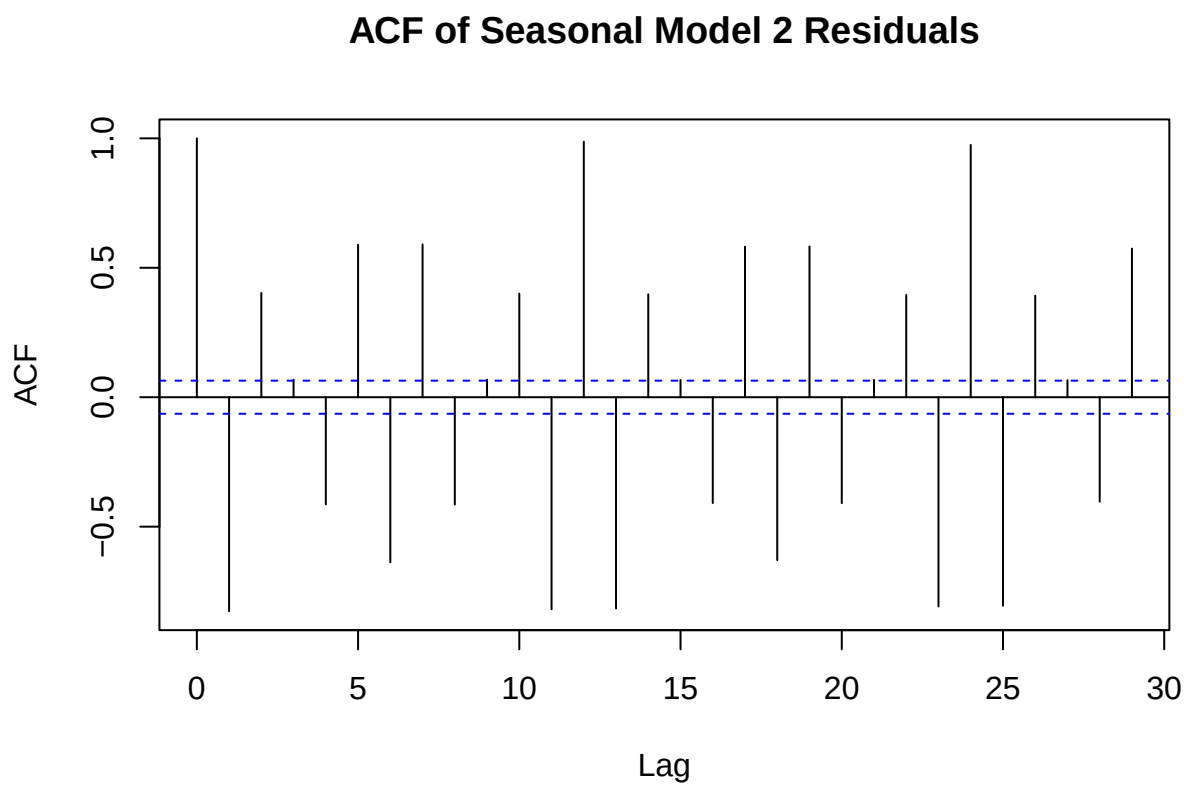
```r
BIC(fit_season1, fit_season2)
```

```
##             df        BIC
## fit_season1  4   319.3231
## fit_season2  8 -1642.0365
```

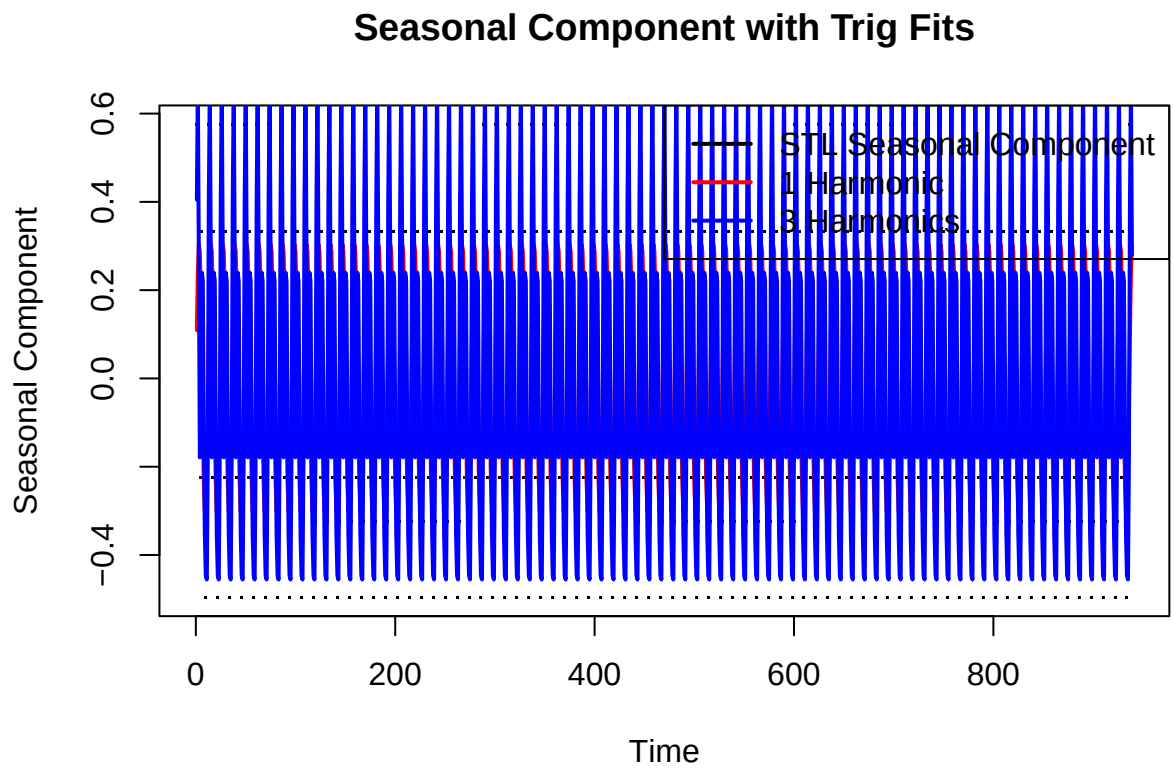
```r
# ACF checks
acf(season_resid1, main = "ACF of Seasonal Model 1 Residuals")
```



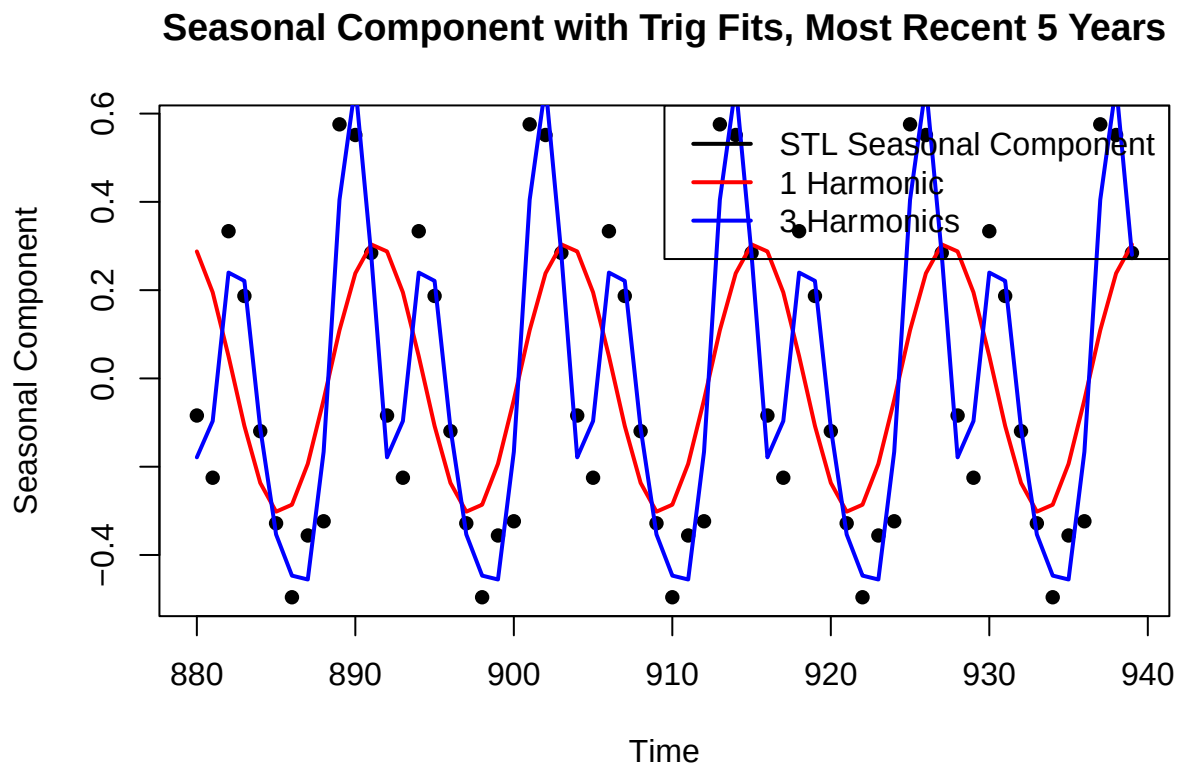

```r
acf(season_resid2, main = "ACF of Seasonal Model 2 Residuals")
```




```r
# full plot
plot(time, season, pch=".", col="black",
     main="Seasonal Component with Trig Fits",
     xlab="Time", ylab="Seasonal Component")

lines(time, season_fit1, col="red", lwd=2)
lines(time, season_fit2, col="blue", lwd=2)

legend("topright",
       legend=c("STL Seasonal Component", "1 Harmonic", "3 Harmonics"),
       col=c("black", "red", "blue"),
       lwd=2)
```



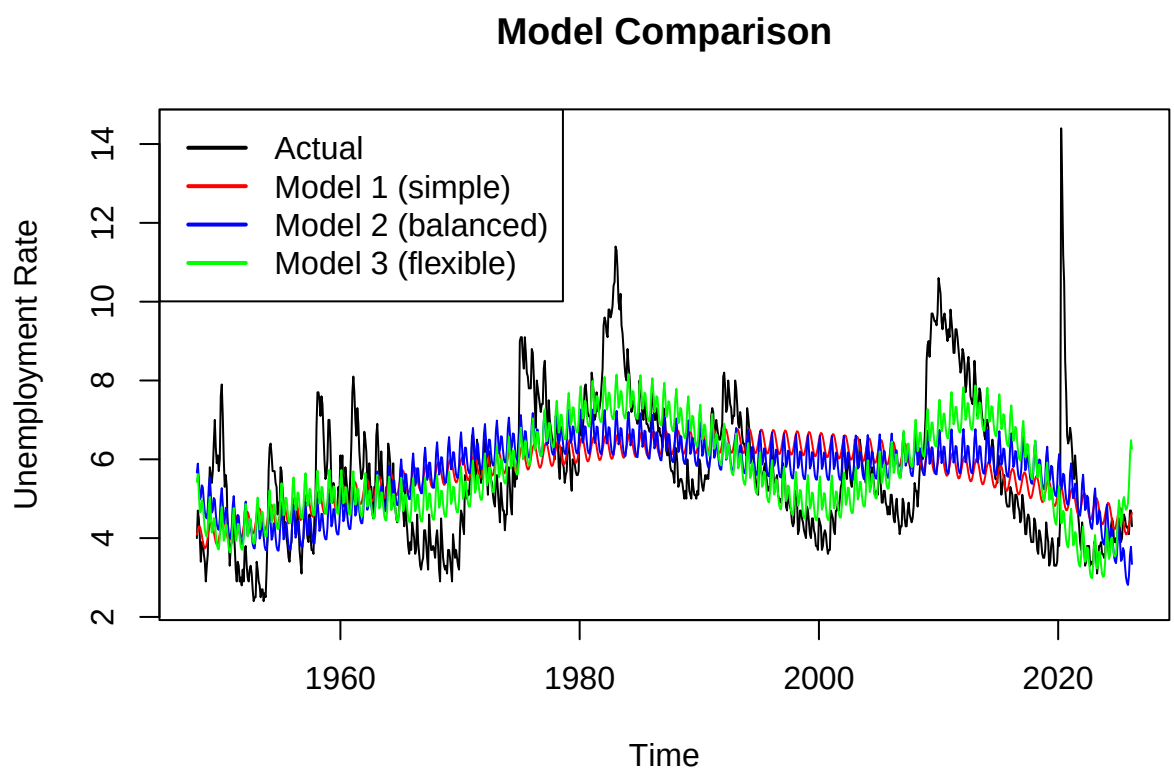

```r
# most recent 60 months / 5 years
n <- length(time)
idx <- (n-59):n

plot(time[idx], season[idx], pch=16, col="black",
     main="Seasonal Component with Trig Fits, Most Recent 5 Years",
     xlab="Time", ylab="Seasonal Component")

lines(time[idx], season_fit1[idx], col="red", lwd=2)
lines(time[idx], season_fit2[idx], col="blue", lwd=2)

legend("topright",
       legend=c("STL Seasonal Component", "1 Harmonic", "3 Harmonics"),
       col=c("black", "red", "blue"),
       lwd=2)
```



Combined Models

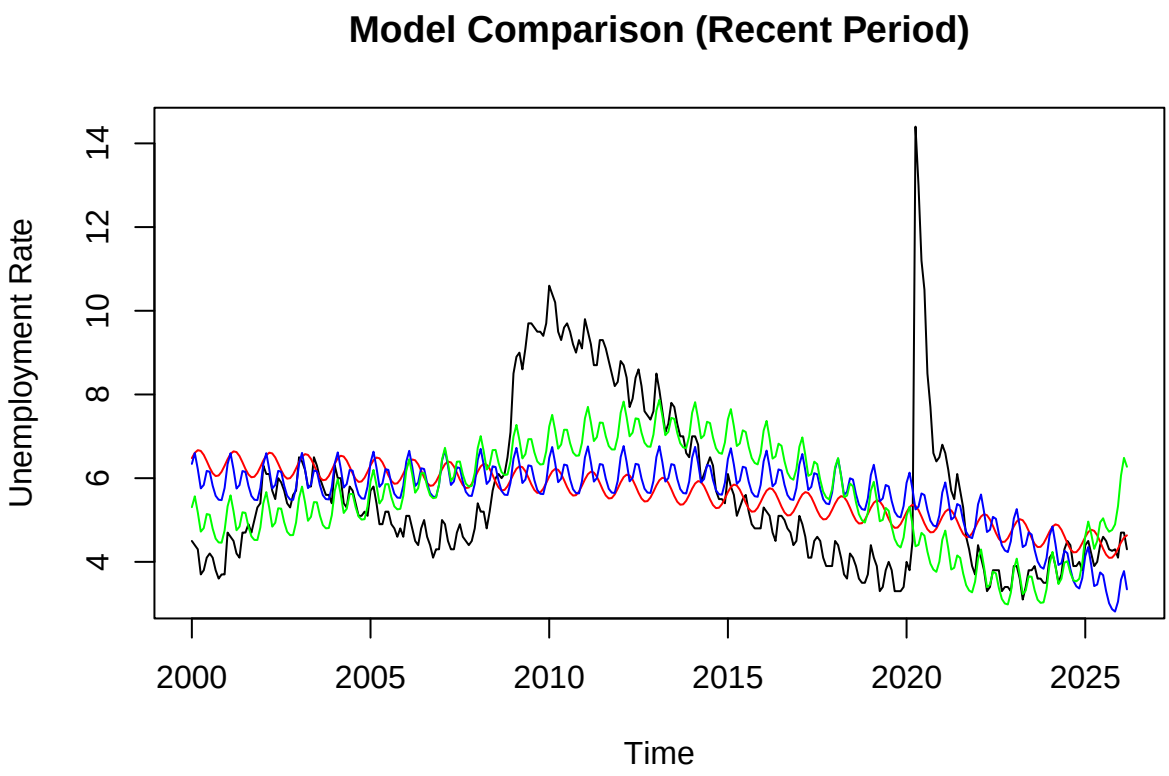
```
#trend comp
tr_poly3 <- fitted(fit_poly3)
tr_poly5 <- fitted(fit_poly5)
tr_poly8 <- fitted(fit_poly8)

#seasonal comp, monthly cycle, cycle() fixed to a 1-12 cycle
month <- cycle(unratensa_ts_interp)

# rebuild seasonal fits###########
coef_s1 <- coef(fit_season1)
coef_s2 <- coef(fit_season2)

ss1 <- coef_s1[1] +
       coef_s1[2]*cos(2*pi*month/12) +
       coef_s1[3]*sin(2*pi*month/12)

ss2 <- coef_s2[1] +
       coef_s2[2]*cos(2*pi*month/12) +
       coef_s2[3]*sin(2*pi*month/12) +
       coef_s2[4]*cos(4*pi*month/12) +
       coef_s2[5]*sin(4*pi*month/12) +
       coef_s2[6]*cos(6*pi*month/12) +
```

```r
       coef_s2[7]*sin(6*pi*month/12)

#combined models
yhat1 <- tr_poly3 + ss1
yhat2 <- tr_poly5 + ss2
yhat3 <- tr_poly8 + ss2

plot(unratensa_ts_interp, col="black",
     main="Model Comparison",
     ylab="Unemployment Rate")

lines(ts(yhat1, start=start(unratensa_ts_interp), frequency=12), col="red")
lines(ts(yhat2, start=start(unratensa_ts_interp), frequency=12), col="blue")
lines(ts(yhat3, start=start(unratensa_ts_interp), frequency=12), col="green")

legend("topleft",
       legend=c("Actual", "Model 1 (simple)", "Model 2 (balanced)", "Model 3 (flexible)"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```

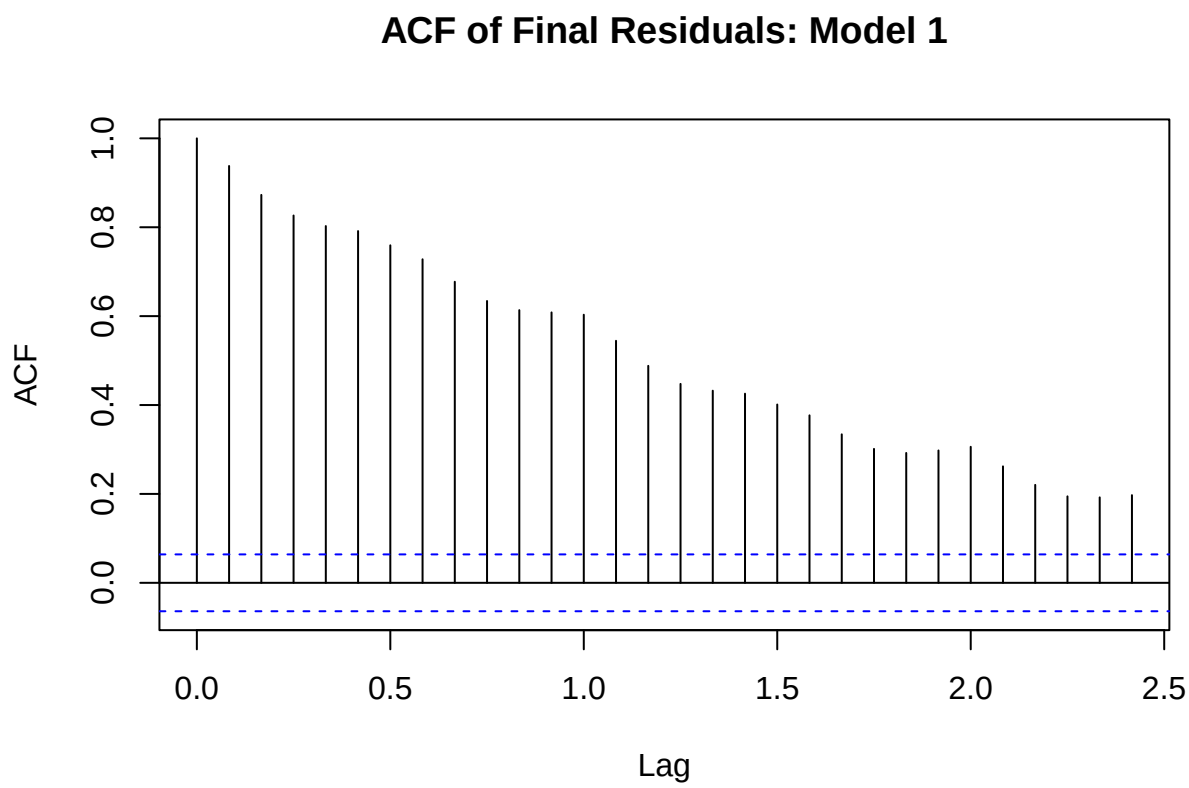



```r
#most recent period
start_plot <- c(2000,1)

plot(window(unratensa_ts_interp, start=start_plot), col="black",
     main="Model Comparison (Recent Period)",
```

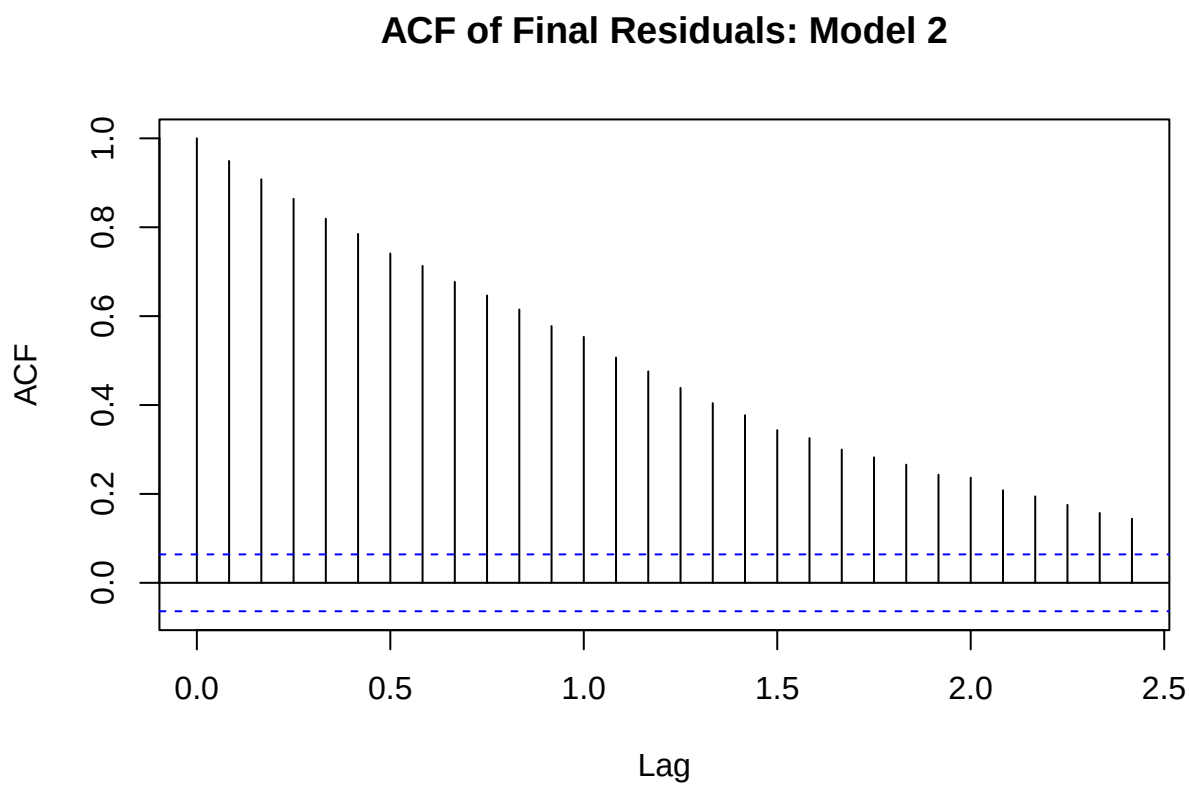
```r
     ylab="Unemployment Rate")

lines(window(ts(yhat1, start=start(unratensa_ts_interp), frequency=12), start=start_plot), col="red")
lines(window(ts(yhat2, start=start(unratensa_ts_interp), frequency=12), start=start_plot), col="blue")
lines(window(ts(yhat3, start=start(unratensa_ts_interp), frequency=12), start=start_plot), col="green")
```



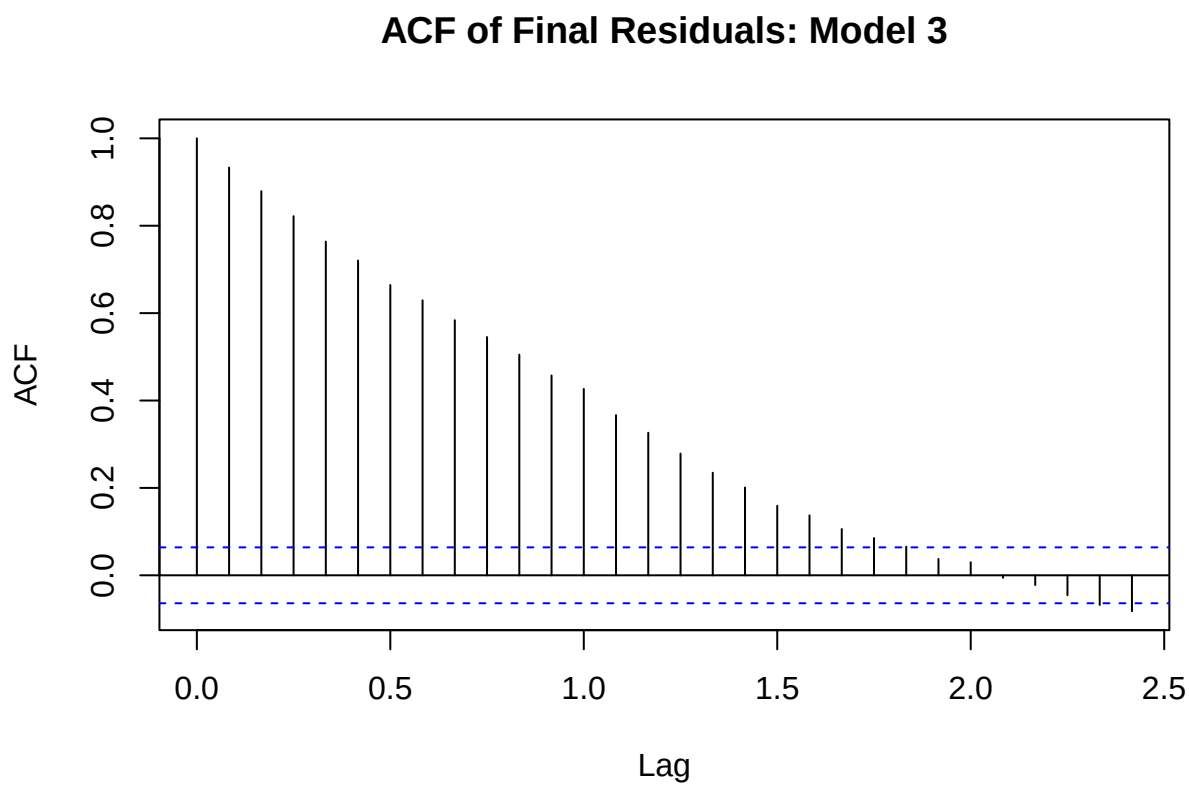

```r
# final residuals
final_resid1 <- unratensa_ts_interp - ts(yhat1, start=start(unratensa_ts_interp), frequency=12)
final_resid2 <- unratensa_ts_interp - ts(yhat2, start=start(unratensa_ts_interp), frequency=12)
final_resid3 <- unratensa_ts_interp - ts(yhat3, start=start(unratensa_ts_interp), frequency=12)

acf(final_resid1, main="ACF of Final Residuals: Model 1")
```




```r
acf(final_resid2, main="ACF of Final Residuals: Model 2")
```




```
acf(final_resid3, main="ACF of Final Residuals: Model 3")
```



Model 3 provides the best regression-based fit because it combines the 8th-degree polynomial trend, which had the lowest AIC/BIC among the trend models, with the 3-harmonic seasonal model, which had much stronger fit statistics than the 1-harmonic model. However, the residual ACF still shows substantial autocorrelation, meaning the regression model captures trend and seasonality but does not fully account for time dependence. This motivates later time series methods such as ARIMA.

II. Time Series Data Analysis (Continued): Smoothing Methods & Forecasting

Naïve Method

The naïve method serves as a simple baseline forecast: each future value is set equal to the most recent observed value. For a non-seasonal series, the forecast for all future horizons h is:

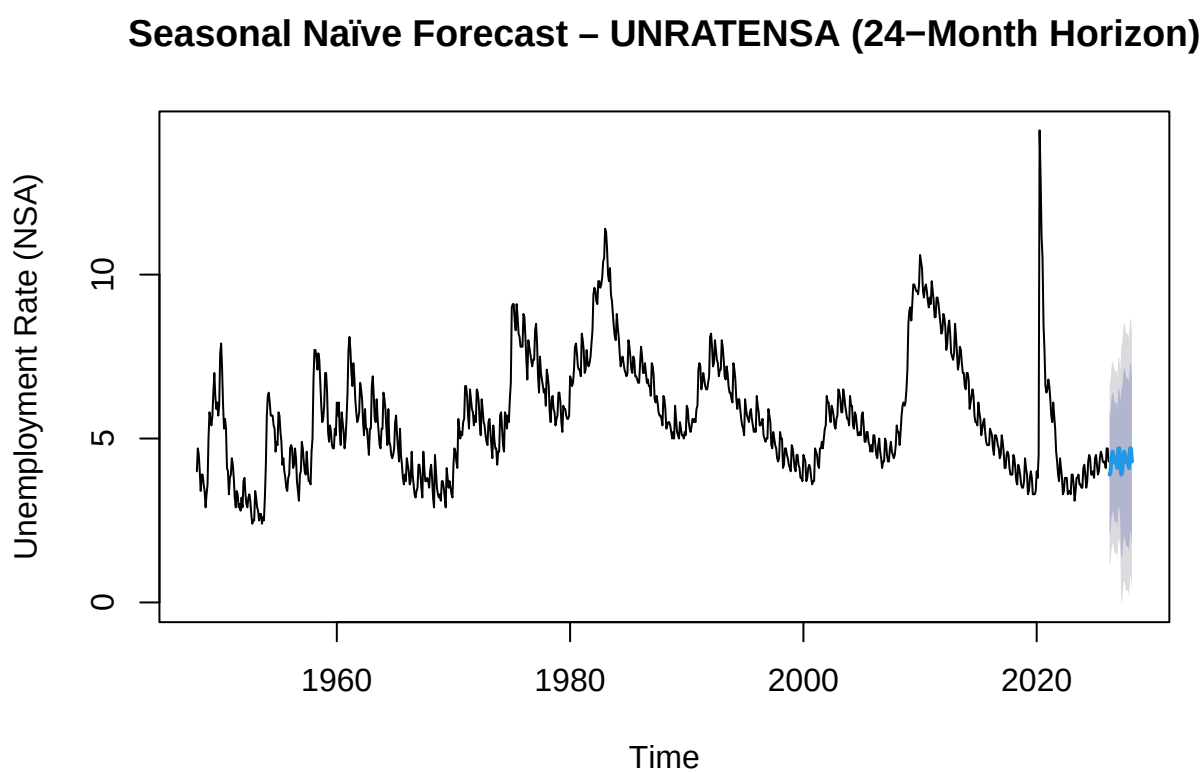
$$\hat{X}_{t+h} = X_t$$

For a seasonal series (seasonal naïve), the forecast repeats the value from the same season in the previous cycle:

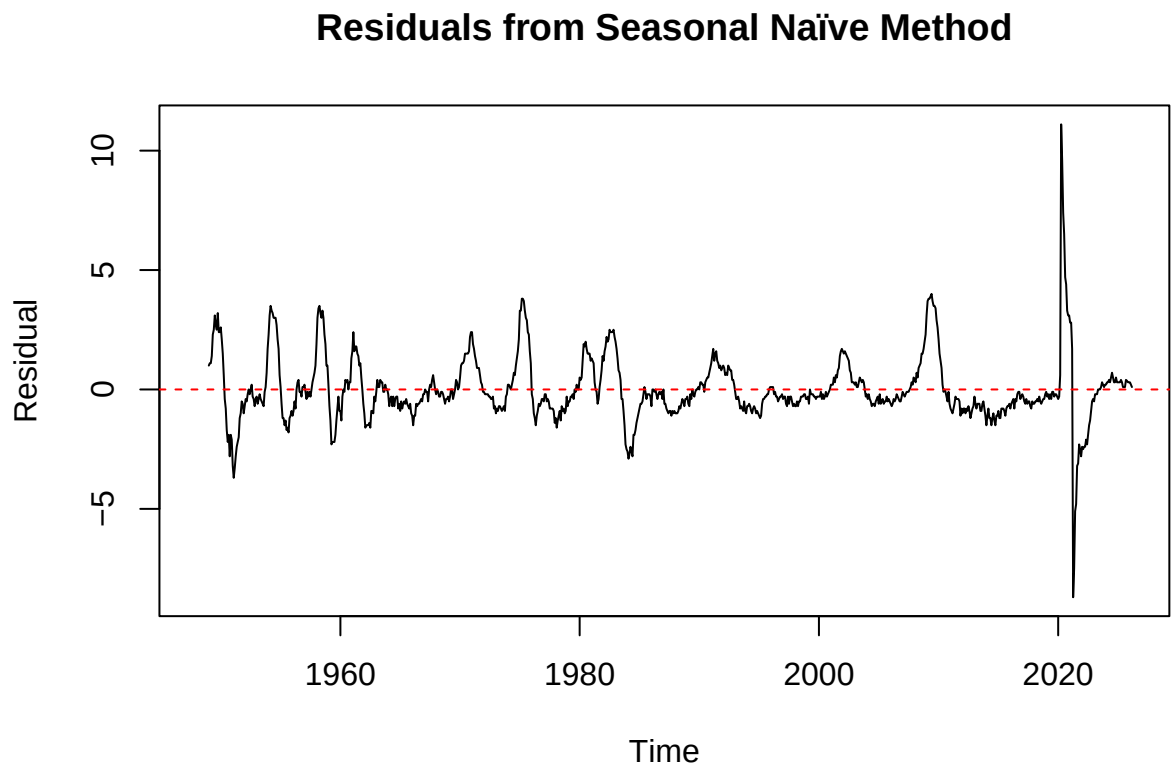
$$\hat{X}_{t+h} = X_{t+h-m}$$

where m is the seasonal period. Because `UNRATENSA` exhibits strong monthly seasonality, the seasonal naïve method is the appropriate baseline.

```r
library(forecast)

# Seasonal naive forecast (12-month seasonality)
naive_fit <- snaive(unratensa_ts_interp, h = 24)

plot(naive_fit,
     main = "Seasonal Naïve Forecast - UNRATENSA (24-Month Horizon)",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
```




```r
# Residual diagnostics for naive baseline
naive_resid <- residuals(naive_fit)
plot(naive_resid,
     main = "Residuals from Seasonal Naïve Method",
     ylab = "Residual",
     xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```



The seasonal naïve forecast simply projects the pattern of the previous year forward, providing a useful benchmark against which more sophisticated methods can be compared.

Exponential Smoothing

Exponential smoothing methods assign geometrically decreasing weights to past observations, giving the most recent data the greatest influence on forecasts. Three variants are applied here, chosen based on the characteristics of `UNRATENSA`.

Simple Exponential Smoothing (SES) SES is appropriate when a series has no consistent trend or seasonality — useful here as a reference. The smoothed estimate at time t is:

$$e_t = \alpha X_t + (1-\alpha)\, e_{t-1}, \quad 0 < \alpha < 1$$

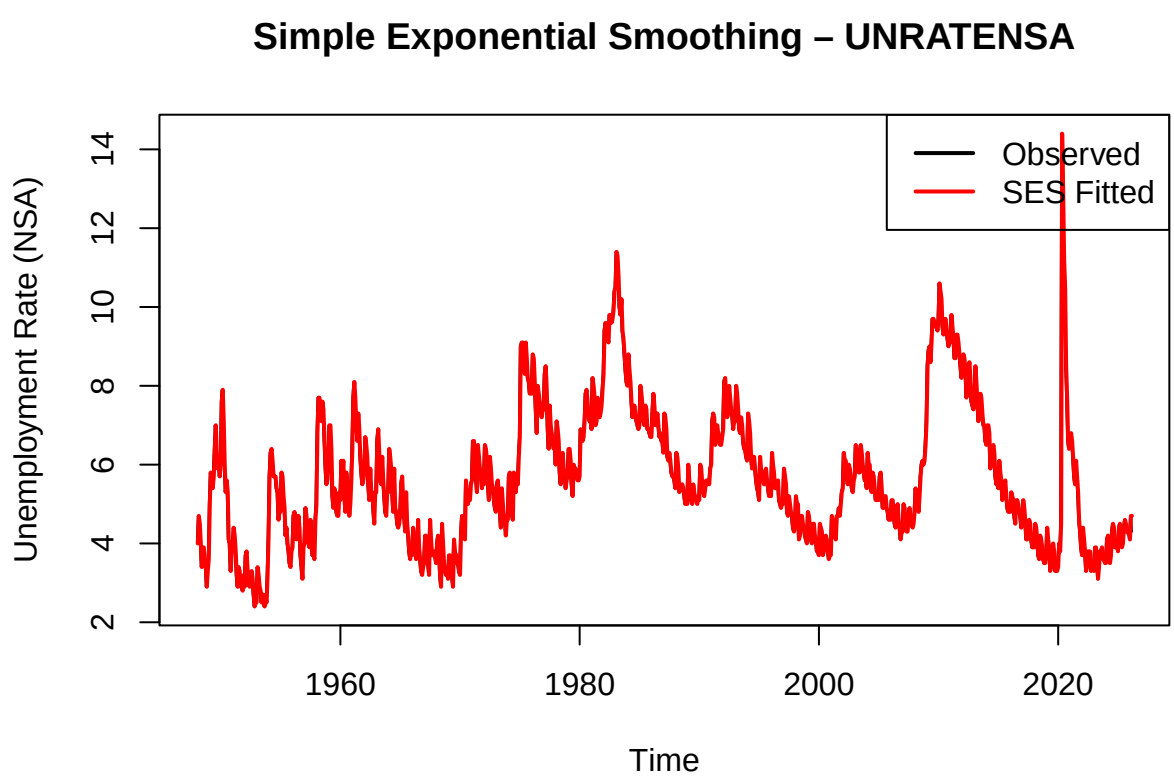
A small α weights historical observations more heavily; a large α reacts quickly to recent changes.

```
# Simple exponential smoothing (no trend, no seasonality -- reference model)
ses_fit <- HoltWinters(unratensa_ts_interp, beta = FALSE, gamma = FALSE)
ses_fit  # print smoothing parameters
```

```
## Holt-Winters exponential smoothing without trend and without seasonal component.
##
```

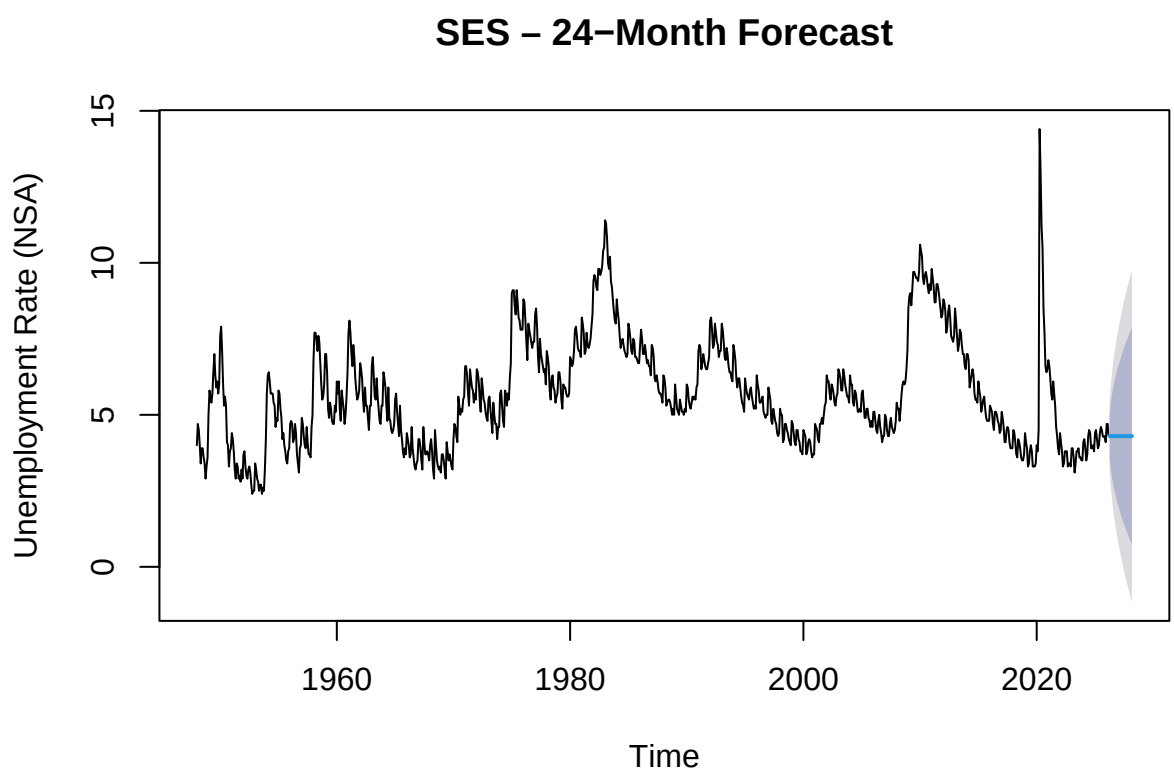
```
## Call:
## HoltWinters(x = unratensa_ts_interp, beta = FALSE, gamma = FALSE)
##
## Smoothing parameters:
##  alpha: 0.9999232
##  beta : FALSE
##  gamma: FALSE
##
## Coefficients:
##       [,1]
## a 4.300031
```

```r
plot(unratensa_ts_interp,
     main = "Simple Exponential Smoothing - UNRATENSA",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
lines(ses_fit$fitted[, 1], col = "red", lwd = 2)
legend("topright",
       legend = c("Observed", "SES Fitted"),
       col    = c("black", "red"),
       lwd    = 2)
```




```r
# 24-month forecast
ses_forecast <- forecast(ses_fit, h = 24)
```

```r
plot(ses_forecast,
     main = "SES - 24-Month Forecast",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time")
```



Holt's Exponential Smoothing (Trend, No Seasonality) Holt's method extends SES by also estimating the slope (trend) of the series using a second smoothing parameter β:

$$\ell_t = \alpha X_t + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1}$$

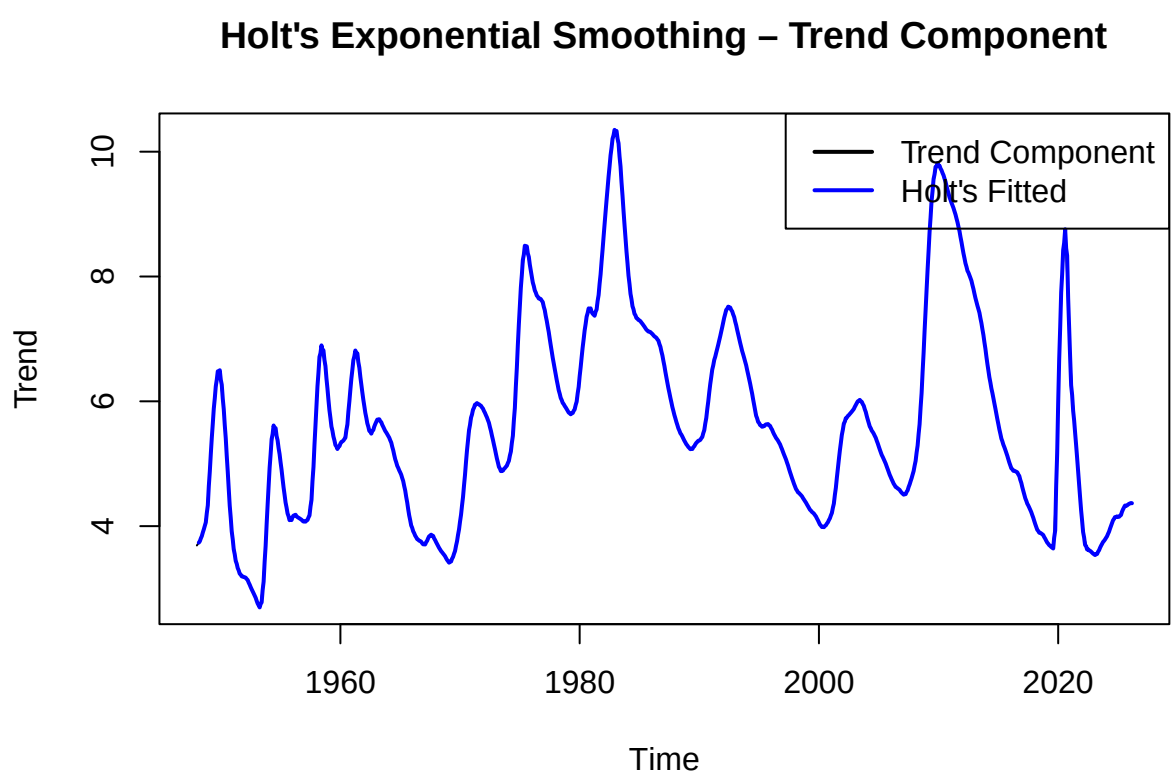
This is applied to the trend component extracted from the STL decomposition.

```r
# Holt's method on the trend component
holts_fit <- HoltWinters(trend, gamma = FALSE)
holts_fit  # print alpha and beta
```

```
## Holt-Winters exponential smoothing with trend and without seasonal component.
##
## Call:
## HoltWinters(x = trend, gamma = FALSE)
##
## Smoothing parameters:
```

```
##  alpha: 0.9184004
##  beta : 1
##  gamma: FALSE
##
## Coefficients:
##           [,1]
## a 4.3655048485
## b 0.0009737063
```

```r
plot(trend,
     main = "Holt's Exponential Smoothing - Trend Component",
     ylab = "Trend",
     xlab = "Time",
     col  = "black")
lines(holts_fit$fitted[, 1], col = "blue", lwd = 2)
legend("topright",
       legend = c("Trend Component", "Holt's Fitted"),
       col    = c("black", "blue"),
       lwd    = 2)
```



Holt-Winters Exponential Smoothing (Trend + Seasonality) Because `UNRATENSA` has both trend and strong monthly seasonality, Holt-Winters (HW) is the most appropriate exponential smoothing method. HW adds a third smoothing parameter γ for the seasonal component:

$$\ell_t = \alpha(X_t - S_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1})$$

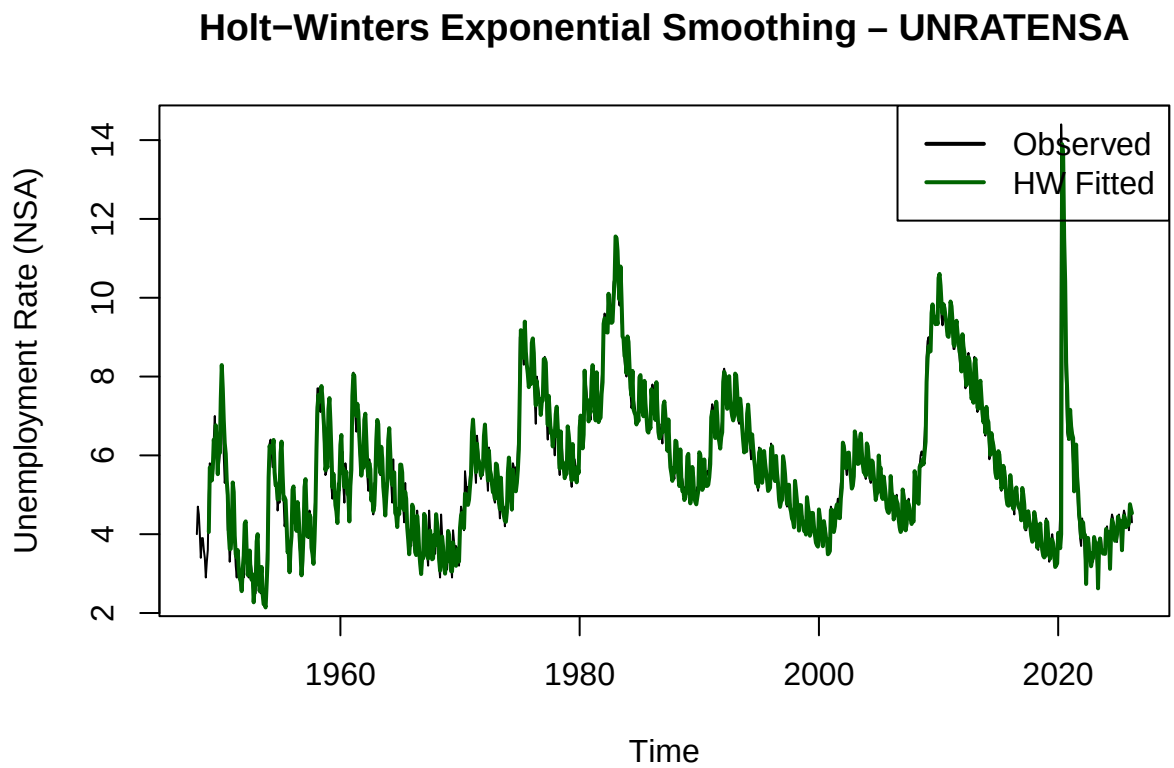
$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1}$$
$$S_t = \gamma(X_t - \ell_t) + (1-\gamma)\, S_{t-m}$$

where $m = 12$ for monthly data.

```r
# Holt-Winters on the full UNRATENSA series
hw_fit <- HoltWinters(unratensa_ts_interp)
hw_fit  # print alpha, beta, gamma
```

```
## Holt-Winters exponential smoothing with trend and additive seasonal component.
##
## Call:
## HoltWinters(x = unratensa_ts_interp)
##
## Smoothing parameters:
##  alpha: 0.9325619
##  beta : 0.01093134
##  gamma: 1
##
## Coefficients:
##             [,1]
## a    4.329301618
## b   -0.008964252
## s1   0.070343008
## s2  -0.194581134
## s3   0.063102535
## s4   0.150125618
## s5  -0.045341574
## s6  -0.353325427
## s7  -0.412697628
## s8  -0.443505478
## s9  -0.469169327
## s10  0.194788226
## s11  0.147037263
## s12 -0.029301618
```

```r
# Plot fitted vs observed
plot(unratensa_ts_interp,
     main = "Holt-Winters Exponential Smoothing - UNRATENSA",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
lines(hw_fit$fitted[, 1], col = "darkgreen", lwd = 2)
legend("topright",
       legend = c("Observed", "HW Fitted"),
       col    = c("black", "darkgreen"),
       lwd    = 2)
```



Forecast Results

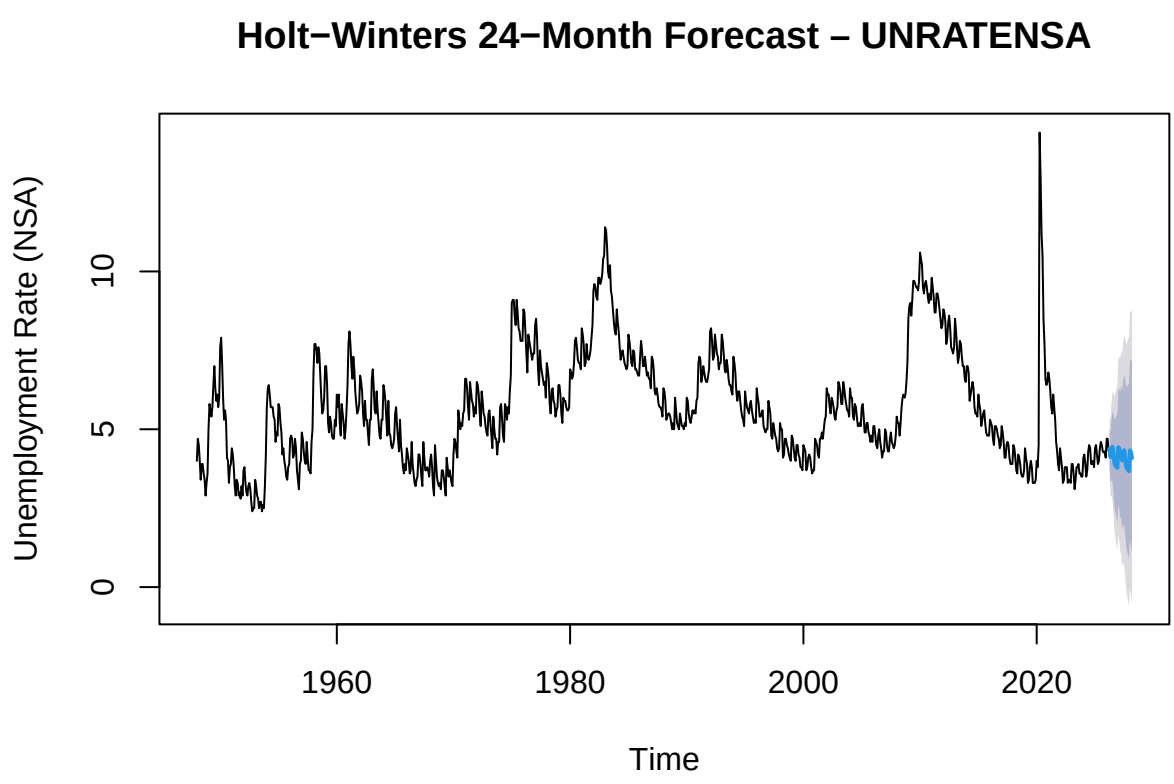
A 24-month ahead forecast is generated from the Holt-Winters model, representing the primary smoothing-based forecast for `UNRATENSA`.

```
hw_forecast <- forecast(hw_fit, h = 24)
hw_forecast  # print forecast table
```

```
##          Point Forecast     Lo 80    Hi 80       Lo 95    Hi 95
## Apr 2026       4.390680 3.8087204 4.972640  3.50064935 5.280711
## May 2026       4.116792 3.3169856 4.916598  2.89359368 5.339990
## Jun 2026       4.365511 3.3922447 5.338778  2.87702847 5.853994
## Jul 2026       4.443570 3.3204402 5.566700  2.72589101 6.161249
## Aug 2026       4.239139 2.9812516 5.497026  2.31536641 6.162911
## Sep 2026       3.922191 2.5402011 5.304180  1.80862005 6.035761
## Oct 2026       3.853854 2.3557432 5.351965  1.56269108 6.145017
## Nov 2026       3.814082 2.2060786 5.422086  1.35485298 6.273311
## Dec 2026       3.779454 2.0665672 5.492341  1.15981975 6.399088
## Jan 2027       4.434447 2.6207978 6.248097  1.66070968 7.208185
## Feb 2027       4.377732 2.4667699 6.288694  1.45516767 7.300297
## Mar 2027       4.192429 2.1870845 6.197773  1.12551921 7.259339
## Apr 2027       4.283109 2.1740781 6.392141  1.05762436 7.508594
```

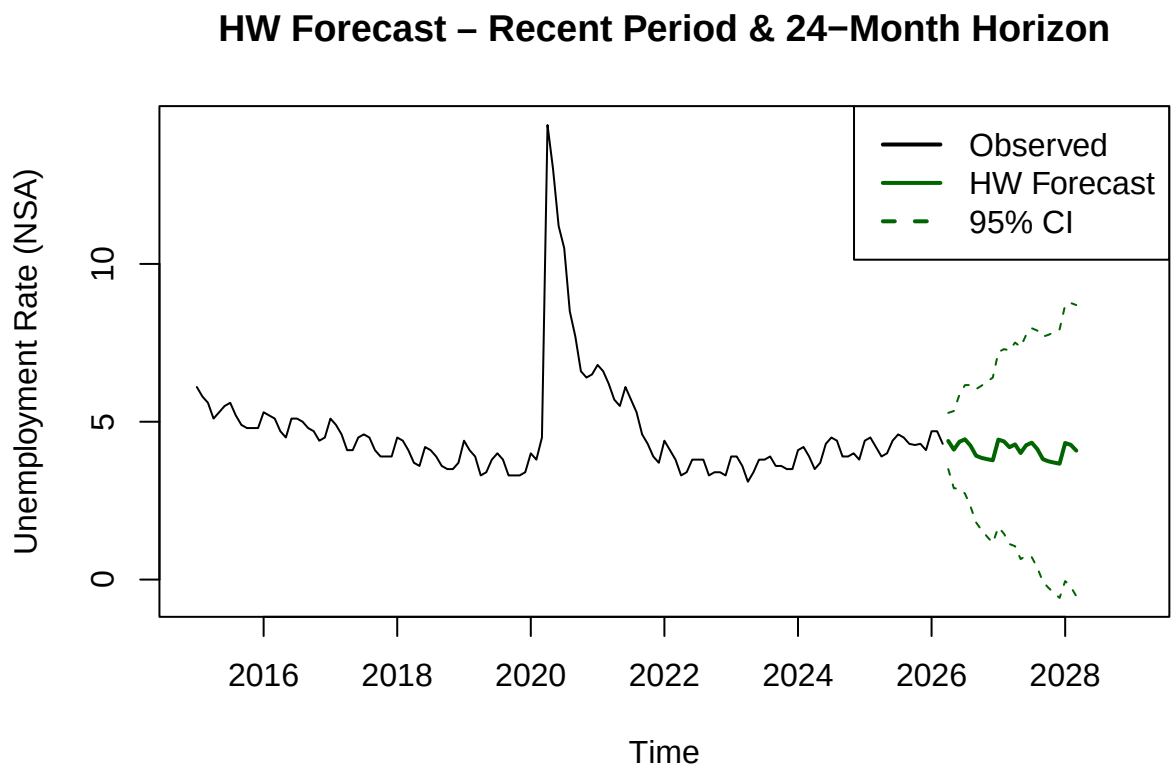
```
## May 2027       4.009221 1.8109918 6.207450  0.64731963 7.371122
## Jun 2027       4.257940 1.9723771 6.543504  0.76247304 7.753408
## Jul 2027       4.335999 1.9647448 6.707254  0.70947856 7.962520
## Aug 2027       4.131568 1.6760788 6.587057  0.37622148 7.886914
## Sep 2027       3.814620 1.2761939 6.353045 -0.06756754 7.696807
## Oct 2027       3.746283 1.1260817 6.366485 -0.26096922 7.753536
## Nov 2027       3.706511 1.0055764 6.407446 -0.42421208 7.837234
## Dec 2027       3.671883 0.8911542 6.452612 -0.58087469 7.924641
## Jan 2028       4.326876 1.4672016 7.186551 -0.04661872 8.700371
## Feb 2028       4.270161 1.3323083 7.208014 -0.22289698 8.763219
## Mar 2028       4.084858 1.0695236 7.100192 -0.52669805 8.696414
```

```r
# Full forecast plot
plot(hw_forecast,
     main = "Holt-Winters 24-Month Forecast - UNRATENSA",
     ylab = "Unemployment Rate (NSA)",
     xlab = "Time",
     col  = "black")
```




```r
# Zoomed view (recent period + forecast)
recent_start <- c(2015, 1)
plot(window(unratensa_ts_interp, start = recent_start),
     xlim = c(2015, end(hw_forecast$mean)[1] + 1),
     ylim = range(c(
       window(unratensa_ts_interp, start = recent_start),
```

```
        hw_forecast$lower, hw_forecast$upper
      ), na.rm = TRUE),
      main = "HW Forecast - Recent Period & 24-Month Horizon",
      ylab = "Unemployment Rate (NSA)",
      xlab = "Time",
      col  = "black")
lines(hw_forecast$mean,   col = "darkgreen", lwd = 2)
lines(hw_forecast$lower[, 2], col = "darkgreen", lty = 2)
lines(hw_forecast$upper[, 2], col = "darkgreen", lty = 2)
legend("topright",
       legend = c("Observed", "HW Forecast", "95% CI"),
       col    = c("black", "darkgreen", "darkgreen"),
       lty    = c(1, 1, 2),
       lwd    = 2)
```



Forecast Accuracy & Residual Diagnostics

```
# Accuracy metrics for each method
cat("=== Seasonal Naive ===\n")
```

```
## === Seasonal Naive ===
```

```r
print(accuracy(naive_fit))
```

```
##                       ME     RMSE      MAE       MPE    MAPE MASE      ACF1
## Training set 0.007403186 1.415716 0.931675 -2.442651 15.9323    1 0.9118357
```

```r
cat("\n=== Simple Exponential Smoothing ===\n")
```

```
##
## === Simple Exponential Smoothing ===
```

```r
ses_fc_train <- forecast(ses_fit, h = 1)
print(accuracy(ses_forecast))
```

```
##                        ME      RMSE       MAE        MPE     MAPE      MASE
## Training set 0.0003198867 0.5679757 0.3586128 -0.4186922 6.494335 0.3849119
##                     ACF1
## Training set 0.05680498
```

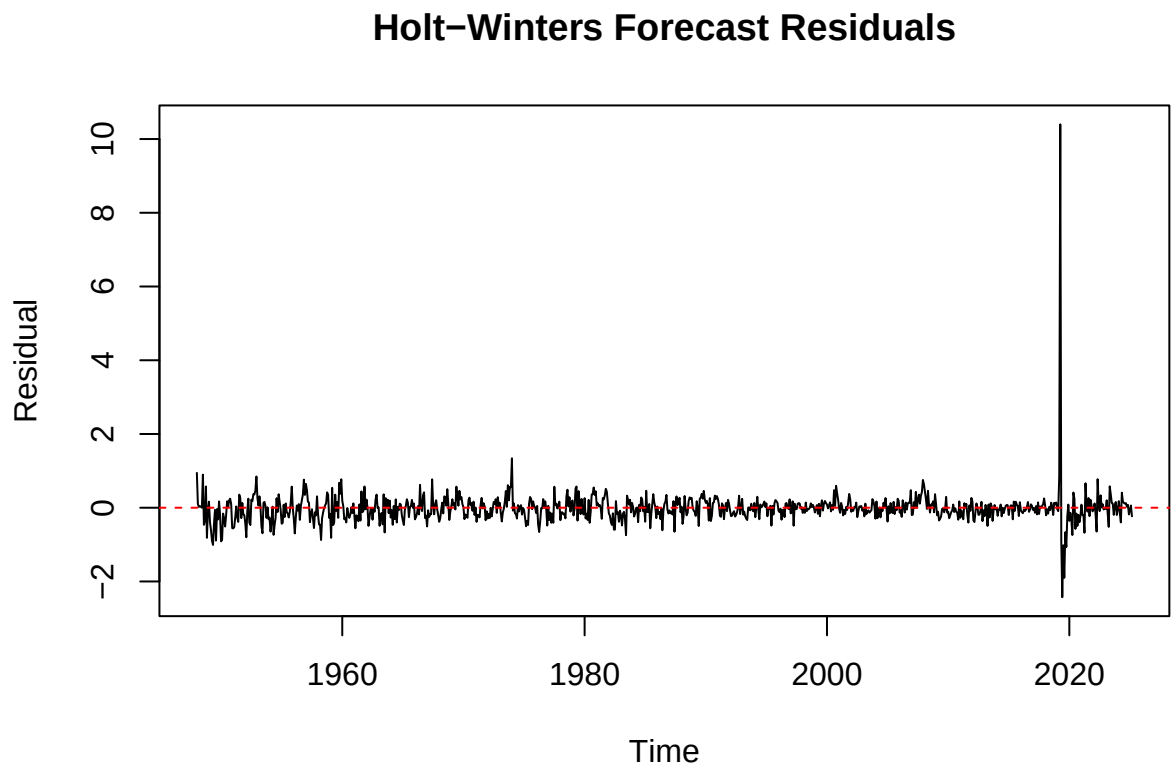
```r
cat("\n=== Holt-Winters ===\n")
```

```
##
## === Holt-Winters ===
```

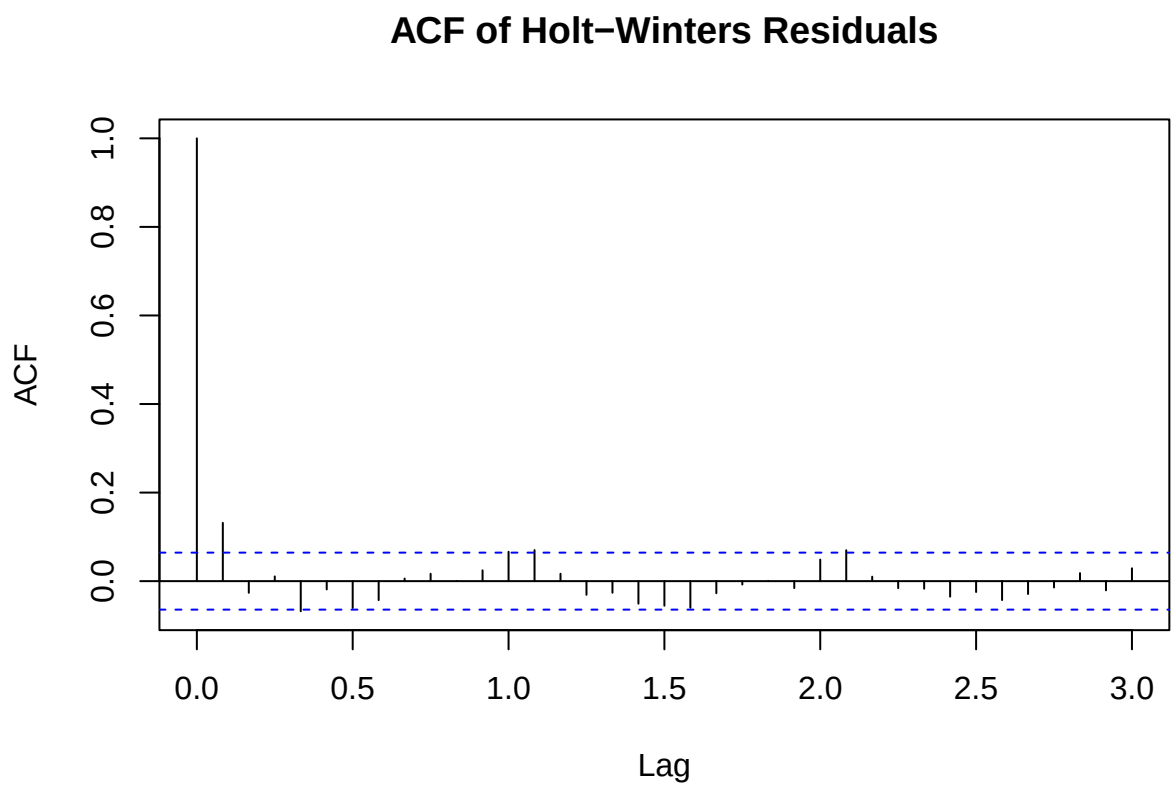
```r
print(accuracy(hw_forecast))
```

```
##                      ME      RMSE       MAE        MPE     MAPE      MASE
## Training set -0.0211553 0.4543536 0.2263107 -0.4714777 4.134302 0.2429073
##                   ACF1
## Training set 0.1315664
```

```r
# Residuals from Holt-Winters
hw_resid <- residuals(hw_forecast)
hw_resid_ts <- ts(hw_resid[!is.na(hw_resid)],
                  start = start(unratensa_ts_interp),
                  frequency = 12)

# Time plot of residuals
plot(hw_resid_ts,
     main = "Holt-Winters Forecast Residuals",
     ylab = "Residual",
     xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```




```r
# ACF of residuals
acf(hw_resid_ts,
    lag.max = 36,
    main    = "ACF of Holt-Winters Residuals")
```




```r
# Ljung-Box test for residual autocorrelation
Box.test(hw_resid_ts, lag = 20, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  hw_resid_ts
## X-squared = 47.617, df = 20, p-value = 0.0004814
```

Interpretation

The Holt-Winters model captures the level, trend, and seasonal structure of `UNRATENSA` simultaneously. The smoothing parameters estimated by `HoltWinters()` indicate:

- α: how quickly the model adapts to changes in the level of unemployment.
- β: how rapidly the estimated trend slope is updated.
- γ: how strongly the seasonal pattern is re-estimated at each cycle.

Values close to zero indicate that the model relies on the full history rather than reacting sharply to recent observations, which is appropriate for a macroeconomic series with gradual structural shifts. The 24-month forecast provides a smoothing-based projection that will be compared against the ARIMA model in the final comparison section.